

p1132R0

out_ptr - a scalable output pointer abstraction

Draft Proposal, 25 June 2018

Authors:

[JeanHeyd Meneide](#)

[Todor Buyukliev](#)

[Isabella Muerte](#)

Audience:

LEWG, LWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Current Source:

github.com/ThePhD/out_ptr/blob/master/papers/source/d1132.bs

Current:

https://rawgit.com/ThePhD/out_ptr/out_ptr/papers/d1132.html

Implementation:

https://github.com/ThePhD/out_ptr

Reply To:

[JeanHeyd Meneide](#)

Abstract

out_ptr is an abstraction to bring both C APIs and smart pointers back into the promised land. It is the fastest, easiest, future-proof, and no-undefined behavior specification of a pointer output parameter that works seamlessly with smart pointers!

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Design Considerations

3.1 Synopsis

3.2 Overview

3.3 Safety

3.4 Casting Support

3.5 Reallocation Support

4 Implementation Experience

4.1 Why Not Wrap It?

5 Performance

5.1 For `std::out_ptr`

5.2 For `std::inout_ptr`

6 Bikeshed

6.1 Alternative Specification

6.2 Naming

7 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial release.

§ 2. Motivation

We have very good tools for handling unique and shared resource semantics, alongside more coming with [Intrusive Smart Pointers](#). Independently between several different companies, studios, and shops - from VMWare and Microsoft to small game development startups -- a common type has been implemented. It has many names: `ptrptr`, `OutPtr`, `PtrToPtr`, `out_ptr`, `WRL::ComPtrRef` and even [unary operator& on CComPtr](#). It is universally focused on one task: making it so a smart pointer can be passed as a parameter to a function which uses an output pointer parameter in C API functions (e.g., `my_type**`).

This paper is a culmination of a private survey of types from the industry to propose a common, future-proof, high-performance `out_ptr` type that is easy to use to make interop with pointer types a little bit simpler and easier for everyone who has ever wanted something like `my_c_function(&my_unique )`; to behave properly.

§ 3. Design Considerations

The core of `out_ptr`'s (and `inout_ptr`'s) design revolves around avoiding the mistakes of the past, preventing continual modification of new smart pointers and outside smart pointers's interfaces to perform the same task, and enabling some degree of performance efficiency without having to wrap every C API function.

§ 3.1. Synopsis

The function template's full specification is:

```

namespace std {
    template <typename Pointer, typename Smart, typename... A>
    auto out_ptr(Smart& s, A&&... a) noexcept
    -> out_ptr_t<Smart, Pointer, std::tuple<A...>>;

    template <typename Smart, typename... A>
    auto out_ptr(Smart& s, A&&... a) noexcept
    -> decltype(out_ptr<PointerOf<Smart>>(s, std::forward<A>(a)...));

    template <typename Pointer, typename Smart, typename... A>
    auto inout_ptr(Smart& s, A&&... a) noexcept
    -> inout_ptr_t<Smart, Pointer, std::tuple<A...>>;

    template <typename Smart, typename... A>
    auto inout_ptr(Smart& s, A&&... a) noexcept
    -> decltype(inout_ptr<PointerOf<Smart>>(s, std::forward<A>(a)...));
}

```

Where `PointerOf` is the `::pointer` type, then `::element_type*`, then `T*` in that order. The return type `out_ptr_t` and its sister type `inout_ptr_t` are a templated types and must at-minimum have the following:

```

template <typename Smart, typename Pointer, typename Tuple>
struct out_ptr_t {
    out_ptr_t(Smart&, Tuple);
    operator Pointer* () noexcept;
    ~out_ptr_t () noexcept;
};

template <typename Smart, typename Pointer, typename Tuple>
struct inout_ptr_t {
    inout_ptr_t(Smart&, Tuple);
    operator Pointer* () noexcept;
    ~inout_ptr_t () noexcept;
};

```

We specify "at minimum" because we expect users to override this type for their own shared, unique, handle-alike, reference-counting, and etc. smart pointers. The destructor of `~out_ptr_t()` calls `.reset()` on the stored smart pointer of type `Smart` with the stored pointer of type `Pointer` and arguments contained in the Tuple-like `Tuple`. `~inout_ptr_t()` does the same, but with the

additional caveat that the constructor for `inout_ptr_t(Smart&, Tuple)` also calls `.release()`, so that a `reset` doesn't double-delete a pointer that the expected re-allocating API used with `inout_ptr` already handles.

§ 3.2. Overview

`out_ptr/inout_ptr` are free functions meant to be used for C APIs:

```
error_num c_api_create_handle(int seed_value, int** p_handle);
error_num c_api_re_create_handle(int seed_value, int** p_handle);
void c_api_delete_handle(int* handle);

struct resource_deleter {
    void operator()( int* handle ) {
        c_api_delete_handle(handle);
    }
};
```

Given a smart pointer, it can be used like so:

```
std::unique_ptr<int, resource_deleter> resource(nullptr);
error_num err = c_api_create_handle(
    24, std::out_ptr(resource)
);
if (err == C_API_ERROR_CONDITION) {
    // handle errors
}
// resource.get() the out-value from the C API function
```

Or, in the re-create (reallocation) case:

```
std::unique_ptr<int, resource_deleter> resource(nullptr);
error_num err = c_api_create_handle(
    24, std::inout_ptr(resource)
);
if (err == C_API_ERROR_CONDITION) {
    // handle errors
}
// resource.get() the out-value from the C API function
```

§ 3.3. Safety

This implementation uses a pack of `...Args` in the signature of `out_ptr` to allow it to be used with other types whose `.reset()` functions may require more than just the pointer value to form a valid and proper smart pointer. This is the case with `std::shared_ptr` and `boost::shared_ptr`:

```
std::shared_ptr<int> resource(nullptr);
error_num err = c_api_create_handle(
    24, std::out_ptr(resource, resource_deleter{})
);
if (err == C_API_ERROR_CONDITION) {
    // handle errors
}
// resource.get() the out-value from
// the C API function
```

Additional arguments past the smart pointer stored in `out_ptr`'s implementation-defined return type will perfectly forward these to whatever `.reset()` or equivalent implementation requires them. If the underlying pointer does not require such things, it may be ignored or discarded (optionally, with a compiler error using a static assert that the argument will be ignored for the given type of smart pointer).

Of importance here is to note that `std::shared_ptr` can and will overwrite any custom deleter present when called with just `.reset(some_pointer);`. Therefore, we make it a compiler error to not pass in a second argument when using `std::shared_ptr` without a deleter:

```
std::shared_ptr<int> resource(nullptr);
error_num err = c_api_create_handle(
    42, std::out_ptr(resource)
); // ERROR: deleter was changed
    // to an equivalent of
    // std::default_delete!
```

It is likely the intent of the programmer to also pass the fictional `c_api_delete_handle` function to this: the above constraint allows us to avoid such programmer mistakes.

§ 3.4. Casting Support

There are also many APIs (COM-style APIs, base-class handle APIs, type-erasure APIs) where the initialization requires that the type passed to the function is of some fundamental (`void**`) or base type that does not reflect what is stored exactly in the pointer. Therefore, it is necessary to sometimes specify what the underlying type `out_ptr` uses is stored as.

It is also important to note that going in the *opposite* direction is also highly desirable, especially in the case of doing API-hiding behind an e.g. `void*` implementation. `out_ptr` supports both scenarios with an optional template argument to the function call.

For example, consider this DirectX Graphics Infrastructure Interface (DXGI) function on `IDXGIFactory6`:

```
HRESULT EnumAdapterByGpuPreference(  
    UINT Adapter,  
    DXGI_GPU_PREFERENCE GpuPreference,  
    REFIID riid,  
    void** ppvAdapter  
);
```

Using `out_ptr`, it becomes trivial to interface with it using an exemplary `std::unique_ptr<IDXGIAdapter, ComDeleter> adapter`:

```
HRESULT result = dxgi_factory.  
EnumAdapterByGpuPreference(0,  
    DXGI_GPU_PREFERENCE_MINIMUM_POWER,  
    IID_IDXGIAdapter,  
    std::out_ptr<void*>(adapter)  
);  
if (FAILED(result)) {  
    // handle errors  
}  
// adapter.get() contains strongly-typed pointer  
);
```

No manual casting, `.release()` fiddling, or `.reset()` is required: the returned type from `out_ptr` handles that.

§ 3.5. Reallocation Support

In some cases, a function given a valid handle/pointer will delete that pointer on your behalf before performing an allocation in the same pointer. In these cases, just `.reset()` is entirely redundant and dangerous because it will delete a pointer that it does not own. Therefore, there is a second abstraction called `inout_ptr`, so aptly named because it is both an input (to be deleted) and an output (to be allocated post-delete). `inout_ptr`'s semantics are exactly like `out_ptr`'s, just with the additional requirement that it calls `.release()` on the smart pointer upon being constructed.

This can be heavily optimized in the case of `unique_ptr`, but to do so from the outside requires Undefined Behavior or modification of the standard library. See [§5.2 For std::inout_ptr](#) for further explication.

§ 4. Implementation Experience

This library has been brewed at many companies in their private implementations, and implementations in the wild are scattered throughout code bases with no unifying type. As noted in [§2 Motivation](#), Microsoft has implemented this in `WRL::ComPtrRef`. Its earlier iteration -- `CComPtr` -- simply overrode `operator&`. We assume they prefer the former after having forced the need with `CComPtr` for `std::addressof`. VMWare has a type that much more closely matches the specification in this paper, titled `Vtl::OutPtr`. The primary author of this paper wrote and used `out_ptr` for over 5 years in their code base working primarily with graphics APIs such as DirectX and OpenGL, and more recently Vulkan. They have also seen a similar abstraction in the places they have interned at.

The primary author of [\[p0468r0\]](#) in pre-r0 days also implemented an overloaded `operator&` to handle interfacing with C APIs, but was quickly talked out of actually proposing it when doing the proposal. That author has joined in on this paper to continue to voice the need to make it easier to work with C APIs without having to wrap the function.

Given that many companies, studios and individuals have all invented the same type independently of one another, we believe this is a strong indicator of agreement on an existing practice that should see a proposal to the standard.

A [full implementation with UB and friendly optimizations is available in the repository](#).

§ 4.1. Why Not Wrap It?

A common point raised while using this abstraction is to simply "wrap the target function". We believe this to be a non-starter in many cases: there are thousands of C API functions and even the most dedicated of tools have trouble producing lean wrappers around them. This tends to work for one-off functions, but suffers scalability problems very quickly.

Templated intermediate wrapper functions which take a function, perfectly forwards arguments, and attempts to generate e.g. a `unique_ptr` for the first argument and contain the boiler plate within itself also causes problems. Besides from the (perhaps minor) concern that such a wrapping function disrupts any auto-completion or tooling, the issue arises that C libraries -- even within themselves -- do not agree on where to place the `some_c_type**` parameter and detecting it properly to write a generic function to automatically do it is hard. Even within the C standard library, some functions have output parameters in the beginning and others have it at the end. The disparity grows when users pick up libraries outside the standard.

§ 5. Performance

Many C programmers in our various engineering shops and companies have taken note that manually re-initializing a `unique_ptr` when internally the pointer value is already present has a measurable performance impact.

Teams eager to squeeze out performance realize they can only do this by relying on type-punning shenanigans to extract the actual value out of `unique_ptr`: this is expressly undefined behavior. However, if an implementation of `out_ptr` could be friended or shipped by the standard library, it can be implemented without performance penalty.

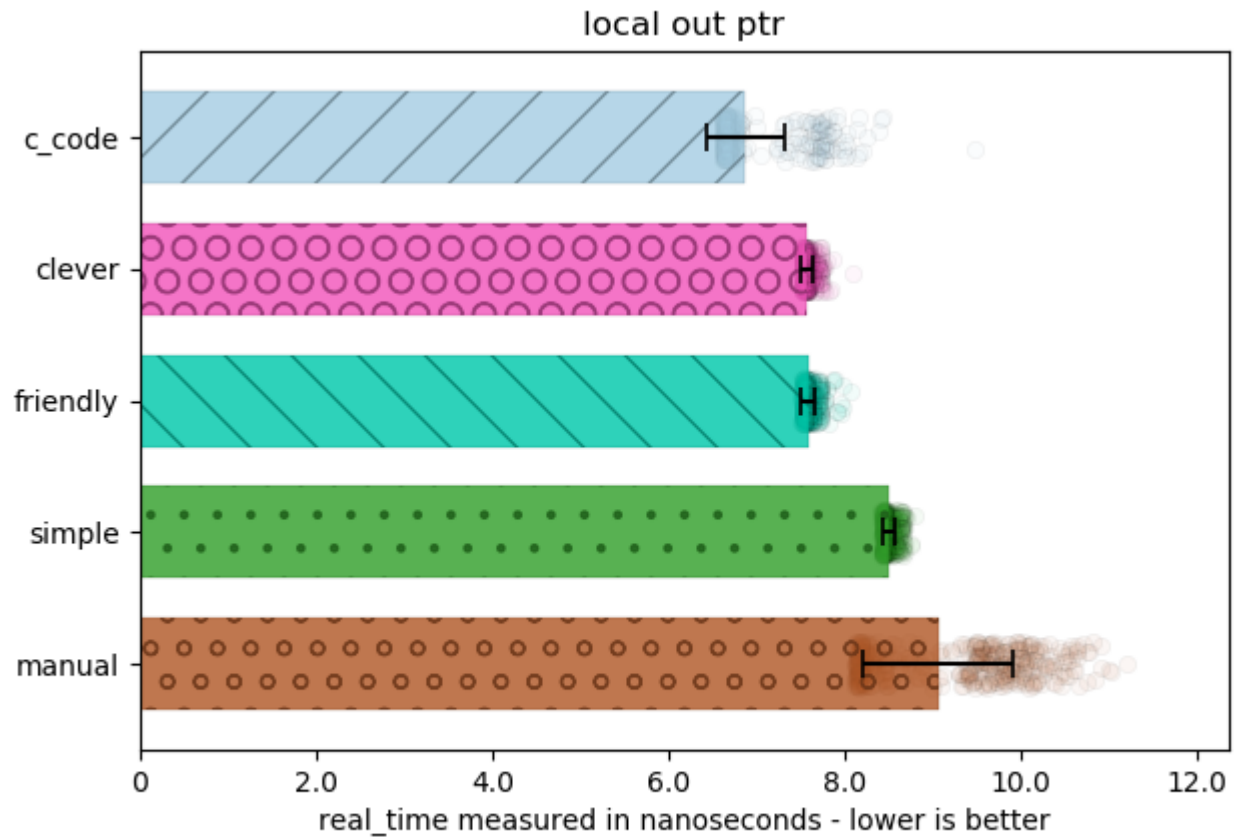
Below are some graphs indicating the performance metrics of the code. 5 categories were measured:

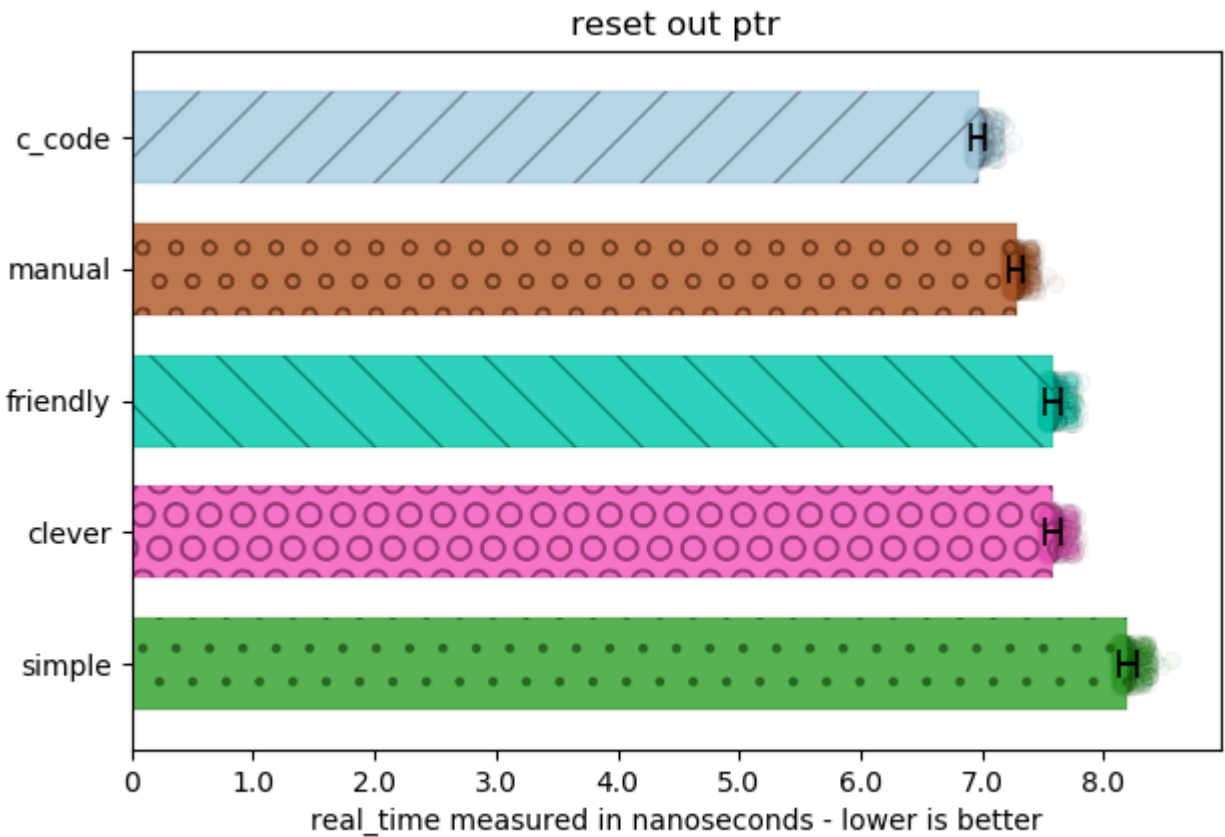
- "c_code": handwritten C code, which does not use this idiom
- "clever": uses UB to alias the pointer value stored in `std::unique_ptr`
- "friendly": modifies VC++'s, libc++'s, and libstdc++'s `std::unique_ptr`s to allow the implementation to friend the `out_ptr` implementation, to access the internals without UB
- "manual": does the work by-hand using reset/release from a `std::unique_ptr`
- "simple": a `out_ptr` implementation that naively resets

The full JSON data for these benchmarks is available [in the repository](#), as well as all of the code necessary to run the benchmarks across all platforms with a simple CMake build system.

§ 5.1. For `std::out_ptr`

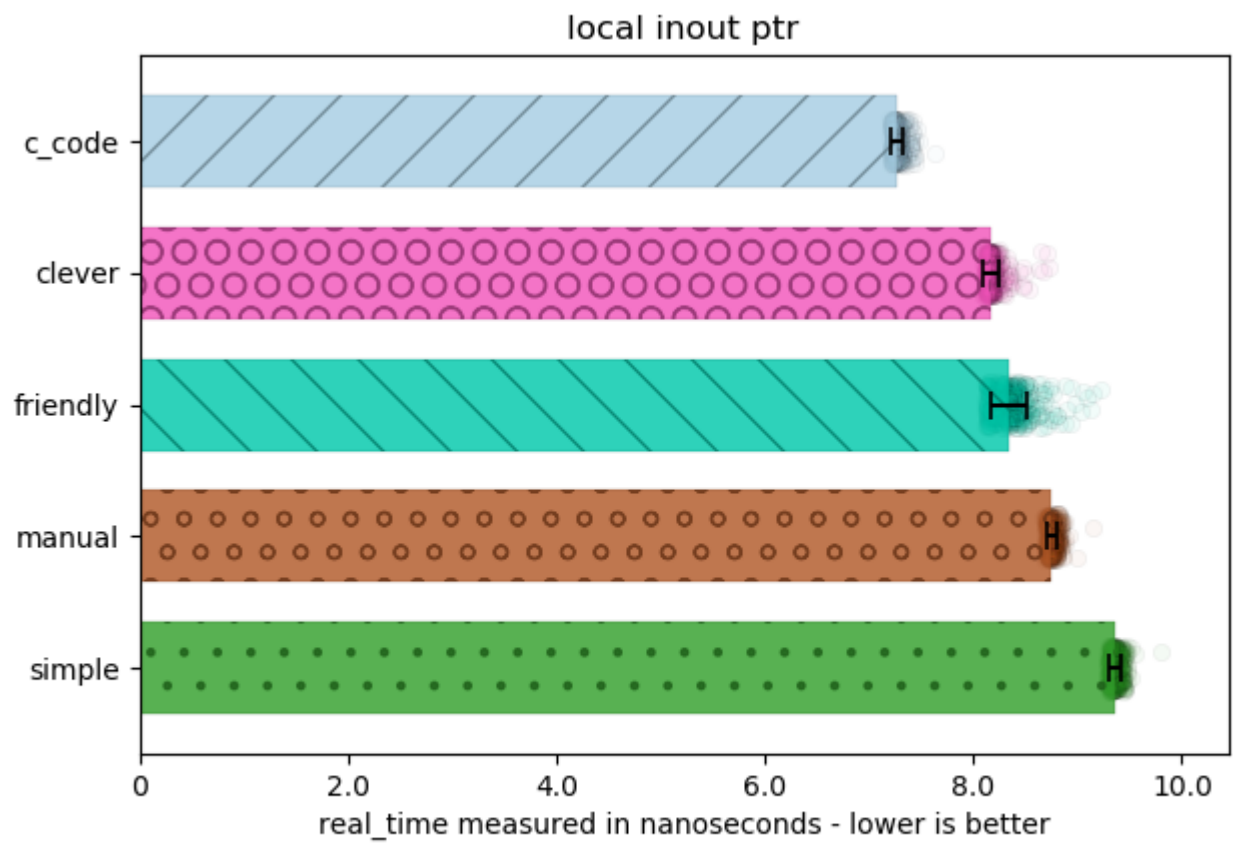
You can observe two graphs for two common `unique_ptr` usage scenarios, which are using the pointer locally and discarding it ("local"), and resetting a pre-existing pointer ("reset") for just an output pointer:

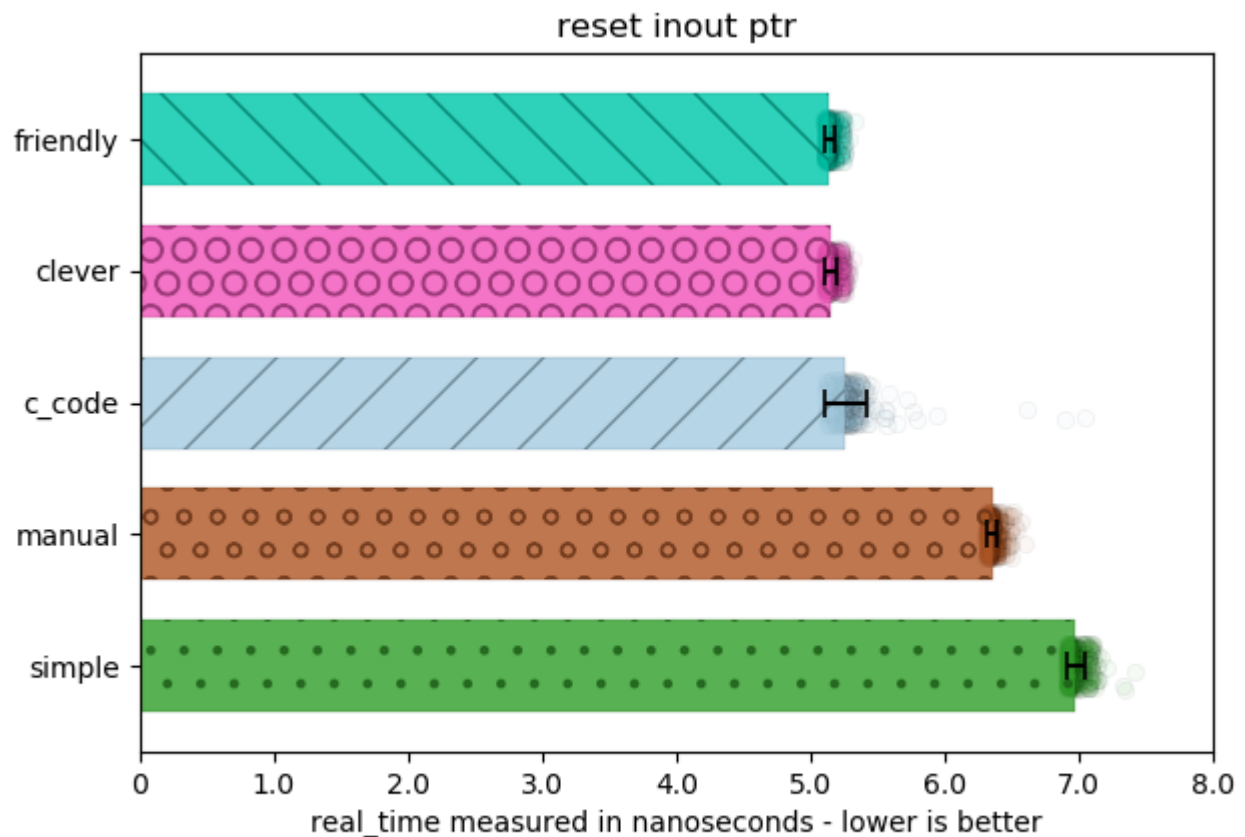




§ 5.2. For `std::inout_ptr`

The speed increase here is even more dramatic: reseating the pointer through `.release()` and `.reset()` is much more expensive than simply aliasing a `std::unique_ptr` directly. Places such as VMWare have to perform Undefined Behavior to get this level of performance with `inout_ptr`: it would be much more prudent to allow both standard library vendors and users to be able to achieve this performance without hacks, tricks, and other I-promise-it-works-I-swear pledges.





§ 6. Bikeshed

As with every proposal, naming, conventions and other tidbits not related to implementation are important. This section is for pinning down all the little details to make it suitable for the standard.

§ 6.1. Alternative Specification

The authors of this proposal know of two ways to specify this proposal's goals.

The first way is to specify both functions `out_ptr` and `inout_ptr` as factories, and then have their types named differently, such as `out_ptr_t` and `inout_ptr_t`. The factory functions and their implementation will be fixed in place, and users would be able to (partially) specialize and customize `std::out_ptr_t` and `std::inout_ptr_t` for types external to the stdlib for maximum performance tweaking and interop with types like `boost::shared_ptr`, `my_lib::local_shared_ptr`, and others. This is the direction this proposal takes.

The second way is to specify the class names to be `std::out_ptr` / `std::inout_ptr`, and then used Template Argument Deduction for Class Templates from C++17 to give a function-like appearance to their usage. Users can still specialize for types external to the standard library. This approach is more Modern C++-like, but contains a caveat. Part of this specification currently is that you can specify the stored pointer for the underlying implementation of `out_ptr` as shown in [§3.4 Casting Support](#). Template Argument Deduction for Class Templates does not allow partial specialization (and for good reason, see the interesting example of `std::tuple<int, int>{1, 2, 3}`).

Therefore, this proposal prefers the approach laid out in [§3.1 Synopsis](#). An alternative would be to use the Deduction Guides approach and have a function with a more explicit name for the casting approach, such as `out_ptr_cast<void*>( ... );` and `inout_ptr_cast<void*>( ... );`.

The authors would like feedback on this specification, in order to make a decision. Please do feel free to e-mail or twitter with discussion, or to have a discussion and link it to the authors.

§ 6.2. Naming

Naming is hard, and therefore we provide a few names to duke it out in the Bikeshed Arena:

For the `out_ptr` part:

- `out_ptr`
- `c_ptr`
- `c_out_ptr`
- `out_c_ptr`
- `out_smart`
- `ptrptr`
- `ptr_to_ptr`
- `ptr_to_smart`
- `ptr_ref`

For the `inout_ptr` part:

- `inout_ptr`
- `c_in_ptr`
- `c_inout_ptr`

- `inout_c_ptr`
- `realloc_c_ptr`
- `inout_smart`,
- `realloc_ptr_to_ptr`
- `realloc_ptr_to_smart`
- `realloc_ptr_ref`

As a pairing, `out_ptr` and `inout_ptr` are the most cromulent and descriptive in the authors' opinion. The type names would follow suit as `out_ptr_t` and `inout_ptr_t`. However, there is an argument for having a name that more appropriately captures the purpose of these abstractions. Therefore, `c_out_ptr` and `c_inout_ptr` would be even better, and the shortest would be `c_ptr` and `c_in_ptr`.

§ 7. Acknowledgements

Thank you to Lounge<C++>'s Cicada, melak47, rmf, and Puppy for reporting their initial experiences with such an abstraction nearly 5 years ago and helping JeanHeyd Meneide implement the first version of this.

Thank you to Mark Zeren for starting this investigation and analysis.

§ References

§ Informative References

[CCOMPTR]

Microsoft. [CComPtr::operator& Operator](https://msdn.microsoft.com/en-us/library/31k6d0k7.aspx). 2015. URL: <https://msdn.microsoft.com/en-us/library/31k6d0k7.aspx>

[P0468R0]

Isabella Muerte. [A Proposal to Add an Intrusive Smart Pointer to the C++ Standard Library](https://wg21.link/p0468r0). 15 October 2016. URL: <https://wg21.link/p0468r0>

[WRL-COMPTRREF]

Microsoft. [ComPtrRef Class](https://docs.microsoft.com/en-us/cpp/windows/comptrref-class). November 4th, 2016. URL: <https://docs.microsoft.com/en-us/cpp/windows/comptrref-class>